Protocollo di comunicazione client-server

Gruppo GC3

Indice

1 Protocollo di comunicazione **2**

2 Messaggi e payload usati dal protocollo **2**

3 Protocollo TCP **4**
3.1 Flusso lato server 4
3.2 Flusso lato client 5
3.3 GSON 6

4 Protocollo RMI **7**
4.1 Setup Connessione 7
4.2 Scambio messaggi 8

5 Sequence Diagrams Commentati **9**
5.1 Boot: avvio del server e connessione dei client 9
5.2 ChooseCard via TCP 11
5.3 ChooseCard via RMI 14

1 Protocollo di comunicazione

Il progetto adotta un'architettura MVC distribuita: view, controller e model si trovano in processi diversi, quindi le interazioni devono necessariamente attraversare la rete. Per mantenere una architettura e pulita, la rete deve essere trasparente per la logica applicativa: né la view né il model ne il controller devono sapere se stanno comunicando via TCP o RMI, per loro deve essere come se si trovino sulla stessa macchina.

Per raggiungere questo obiettivo il sistema introduce due contratti che simulano localmente la presenza della controparte remota — ciascuno finge di essere l'oggetto che il proprio lato dovrebbe avere vicino:

- `VirtualView` (risiede sul server): per il model e per il controller è la view del client. Come una view locale contatta il controller quando riceve un input utente, `VirtualView` riceve i comandi dal client e li instrada al controller appropriato tramite `MessageRouter`. Come una view locale viene aggiornata dal model, il model la registra come `Listener` tramite l'Observer pattern e chiama `notifyUpdate()` su di essa esattamente come farebbe con una view locale; è la `VirtualView` concreta che si occupa di serializzare e trasportare l'aggiornamento attraverso la rete verso il client reale.

- `VirtualServer` (risiede sul client): per la UI è il lato ricevente del server. Riceve i messaggi che arrivano dalla rete e li consegna alla view locale, che non sa nulla del trasporto sottostante.

Questi contratti hanno un doppio ruolo: a livello architetturale sono i punti di comunicazione tra client e server; a livello implementativo permettono a TCP e RMI di offrire gli stessi servizi funzionali, senza che la logica applicativa debba sapere quale protocollo è in uso.

Tuttavia, TCP e RMI hanno vincoli tecnici diversi che impediscono di usare le stesse interfacce in entrambi i casi. Nel caso TCP le interfacce sono normali interfacce Java. Nel caso RMI, invece, le interfacce remote devono estendere `Remote` e i loro metodi devono dichiarare `RemoteException`, perché RMI non può garantire che una chiamata remota abbia successo come farebbe una chiamata locale.

Questa differenza introduce nel codice due livelli di contratto:

- **Livello applicativo** — `VirtualView` e `VirtualServer` descrivono i servizi indipendentemente dal protocollo usato;

- **Livello RMI** — `RmiServerService` e `RmiClientService` espongono servizi equivalenti con i vincoli tecnici richiesti da RMI.

L'allineamento funzionale tra i due livelli è quindi:

- `VirtualView` ↔ `RmiServerService`: comandi client → server;

- `VirtualServer` ↔ `RmiClientService`: notifiche server → client.

RMI richiede un ulteriore contratto, `RmiConnectionService`, viene usato dal client per aprire la connessione iniziale, ottenere lo stub della propria sessione server-side e passare al server il proprio callback per le notifiche. È il punto di ingresso di tutta la comunicazione RMI, prima che esistano sessioni individuali per ogni client.

2 Messaggi e payload usati dal protocollo

Nel codice bisogna distinguere tra messaggio e payload. Il payload contiene i dati applicativi veri e propri: il nickname scelto dal client, l'id della lobby, la carta selezionata, lo stato aggiornato

della partita. Il messaggio è la busta che aggiunge le informazioni necessarie a interpretare quel payload.

La forma concreta che hanno questi tipi è dettata principalmente dal problema di TCP. Sulla socket arriva solo una stringa JSON: il ricevente non sa a priori quale classe concreta di payload ricostruire, perché a livello di contratto il payload è visto solo tramite le interfacce `ClientPayload` o `ServerPayload`. Per risolvere questo problema ogni messaggio include un campo `type`, che fa da discriminante: il ricevente legge prima il tipo, poi chiede a GSON di deserializzare il payload nella classe concreta corrispondente.

Il wrapper esiste quindi in due varianti, una per ciascuna direzione:

- `ClientMessage`, con un `ClientMessageType` e un payload che implementa `ClientPayload`, per i messaggi client → server;
- `ServerMessage`, con un `ServerMessageType` e un payload che implementa `ServerPayload`, per i messaggi server → client.

RMI non ha questo problema: gli oggetti vengono passati direttamente tramite invocazione remota, senza serializzazione JSON esplicita. Tuttavia riutilizza gli stessi payload, così TCP e RMI condividono lo stesso contratto dati a livello applicativo e la logica di dominio non dipende dal protocollo scelto.

I payload sono organizzati gerarchicamente rispetto alla fase di gioco:

- pre-lobby: payload usati nella fase di autenticazione e scoperta delle lobby;
- lobby: payload usati nella fase di creazione e gestione della lobby;
- in-game: payload usati durante la partita.

E rispetto al senso di comunicazione:

- client → server: package `clienttoserver`, `ClientPayload`;
- server → client: package `servertoclient`, `ServerPayload`.

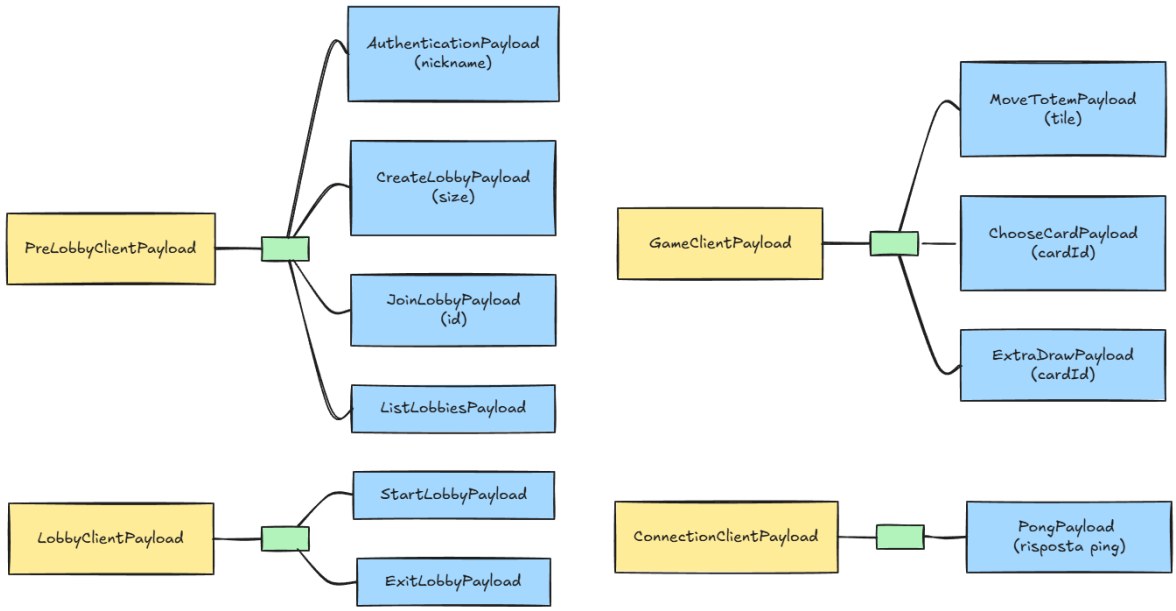


Figura 1: Gerarchia dei payload client → server

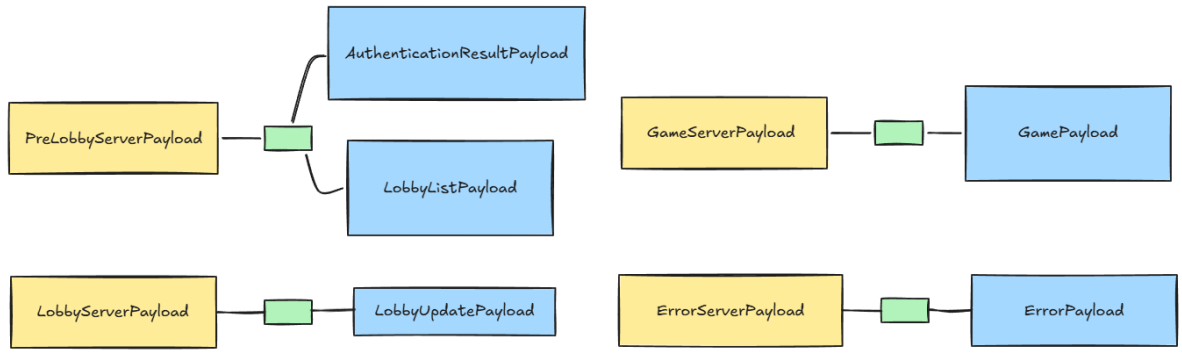


Figura 2: Gerarchia dei payload server → client

Alcuni payload, come `LobbySummaryPayload`, `BoardPayload`, `PlayerPayload`, `CardPayload`, `OfferSpacePayload`, `StartingBlockPayload`, `TribePayload`, `TurnOrderPayload` e `TurnStatePayload`, non vengono inviati direttamente come messaggi principali ma sono payload di supporto annidati dentro payload più grandi, usati per strutturare dati complessi come lo stato della board o il turno corrente.

3 Protocollo TCP

TCP è un protocollo di trasporto che si basa su un canale di comunicazione bidirezionale: la socket, identificata univocamente dalla quadrupla `(IP_sorgente, porta_sorgente, IP_destinazione, porta_destinazione)`. Una volta stabilita la connessione, i due endpoint possono scrivere e leggere dati sullo stesso canale in entrambe le direzioni.

Dal punto di vista dei dati, però, la socket espone uno stream continuo di byte: non esiste un confine naturale tra un messaggio e il successivo. Nel progetto la scelta è semplice: ogni messaggio applicativo viene serializzato in JSON e scritto sulla socket come una singola riga di testo. Dall'altro lato il messaggio viene letto con `readLine()`, quindi una riga corrisponde a un messaggio completo.

La lettura con `readLine()` è bloccante: il thread rimane sospeso finché non arriva una riga completa o la connessione non si chiude. Per questo motivo la lettura non può avvenire sul thread principale: sia il server che il client dedicano un thread separato alla ricezione dei messaggi, così il resto dell'applicazione può continuare a operare mentre il thread di lettura è in attesa.

```
while ((message = reader.readLine()) != null) {
    handleMessage(message);
}
```

La scrittura avviene invece solo quando c'è un comando o una notifica da inviare, quindi non richiede un thread dedicato:

```
writer.println(serializedMessage);
```

3.1 Flusso lato server

Sul server viene creato un `ConnectionAcceptor`, che apre la `ServerSocket` sulla porta configurata e gira su un thread dedicato in attesa di nuove connessioni:

```
ConnectionAcceptor acceptor = new ConnectionAcceptor(tcpPort,
    serverConnectionLifecycle, router);
Thread acceptorThread = new Thread(acceptor, "mesos-connection-acceptor");
```

```
acceptorThread.start();
```

Per ogni client che si connette, il `ConnectionAcceptor` crea una `VirtualViewTCP` dedicata e la avvia su un thread proprio:

```
Socket clientSocket = this.serverSocket.accept();
VirtualViewTCP virtualView = new VirtualViewTCP(clientSocket, this.
    serverConnectionLifecycle, this.router);
Thread virtualViewThread = new Thread(virtualView, "mesos-virtual-view-" +
    clientSocket.getPort());
virtualViewThread.start();
```

Ogni `VirtualViewTCP` è quindi la sessione di un singolo client: gira sul proprio thread, legge i messaggi in arrivo da quel client e può inviare notifiche verso di lui.

3.2 Flusso lato client

Sul client, `ServerProxyTCP` apre la socket verso il server e prepara il writer per i messaggi in uscita. Contestualmente avvia un `VirtualServerTCP` su un thread dedicato, che si occupa di restare in ascolto dei messaggi in ingresso dal server:

```
this.socket = new Socket(serverHost, port);
this.writer = new PrintWriter(this.socket.getOutputStream(), true);
this.virtualServer = new VirtualServerTCP(this.socket, clientView, this);
this.virtualServerThread = new Thread(this.virtualServer, "mesos-client-server-message
    -receiver");
this.virtualServerThread.start();
```

Dentro `VirtualServerTCP` viene preparato il reader e parte il ciclo di lettura:

```
BufferedReader reader = new BufferedReader(new InputStreamReader(this.socket.
    getInputStream()));

while (this.running && (serializedMessage = reader.readLine()) != null) {
    this.handleMessage(serializedMessage);
}
```

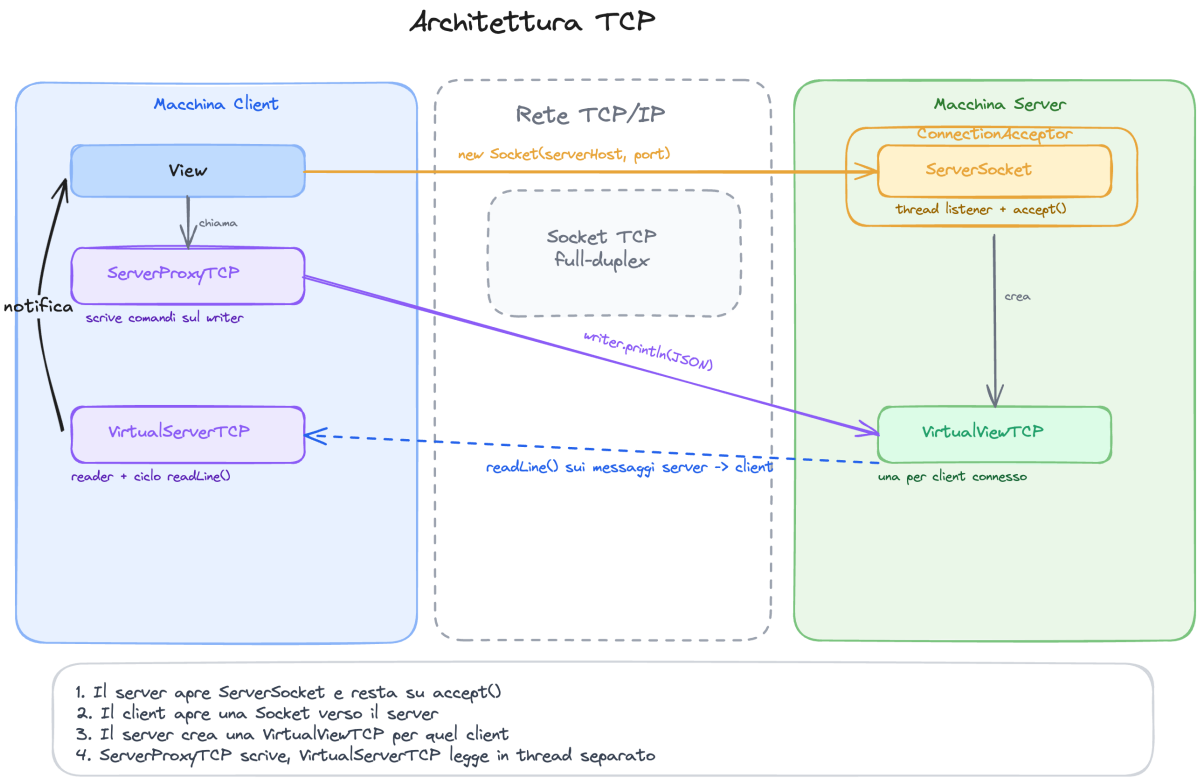


Figura 3: Flusso TCP

In sintesi il flusso concreto TCP è:

- il server avvia il `ConnectionAcceptor` su un thread dedicato;
- per ogni client che si connette viene creata una `VirtualViewTCP` su un proprio thread;
- il client apre la socket e avvia `VirtualServerTCP` su un thread dedicato alla ricezione;
- client → server: `ServerProxyTCP` serializza e scrive sul writer;
- server → client: `VirtualViewTCP` serializza e scrive, `VirtualServerTCP` legge e consegna alla view.

3.3 GSON

GSON è la libreria usata per trasformare oggetti Java in stringhe JSON e viceversa. La serializzazione di un messaggio in uscita è diretta:

```
ClientMessage message = new ClientMessage(ClientMessageType.AUTHENTICATE, new
    AuthenticationPayload("tom"));
String serializedMessage = GSON.toJson(message);
```

La deserializzazione in ingresso è più articolata perché il campo `payload` è dichiarato come interfaccia (`ClientPayload` o `ServerPayload`) e GSON non sa quale classe concreta istanziare. Il `type` fa quindi da discriminante: il messaggio viene prima letto come `JsonObject` per estrarre il tipo, poi il payload viene deserializzato nella classe concreta corrispondente. Questo processo è visibile nel `ClientCommandHandler` lato server:

```
JsonObject root = GSON.fromJson(rawMessage, JsonObject.class);
ClientMessageType type = ClientMessageType.valueOf(root.get("type").getAsString());
```

```
JsonElement payloadElement = root.get("payload");

ClientPayload payload = switch (type) {
    case AUTHENTICATE -> GSON.fromJson(payloadElement, AuthenticationPayload.class);
    case CREATE_LOBBY  -> GSON.fromJson(payloadElement, CreateLobbyPayload.class);
    case MOVE_TOTEM   -> GSON.fromJson(payloadElement, MoveTotemPayload.class);
    // altri type mappati sulle rispettive classi di payload
};
```

4 Protocollo RMI

RMI è un'astrazione Java per la comunicazione distribuita: permette a una JVM di invocare metodi su oggetti che risiedono in un'altra JVM, con la stessa sintassi di una chiamata locale. A differenza di TCP, il codice applicativo non apre socket, non serializza righe JSON e non mantiene cicli di `readLine()`: questi aspetti restano presenti a livello di trasporto ma sono nascosti dal meccanismo di RMI.

La differenza architetturale rispetto a TCP riguarda soprattutto la direzione server → client. In TCP il server può scrivere al client in qualsiasi momento perché lo stream bidirezionale è sempre aperto. In RMI non esiste uno stream: tutto è invocazione di metodi su oggetti remoti. Per notificare il client, il server ha bisogno di qualcosa da poter chiamare: per questo il client deve esportare un oggetto remoto, la callback, e passarlo al server durante la connessione.

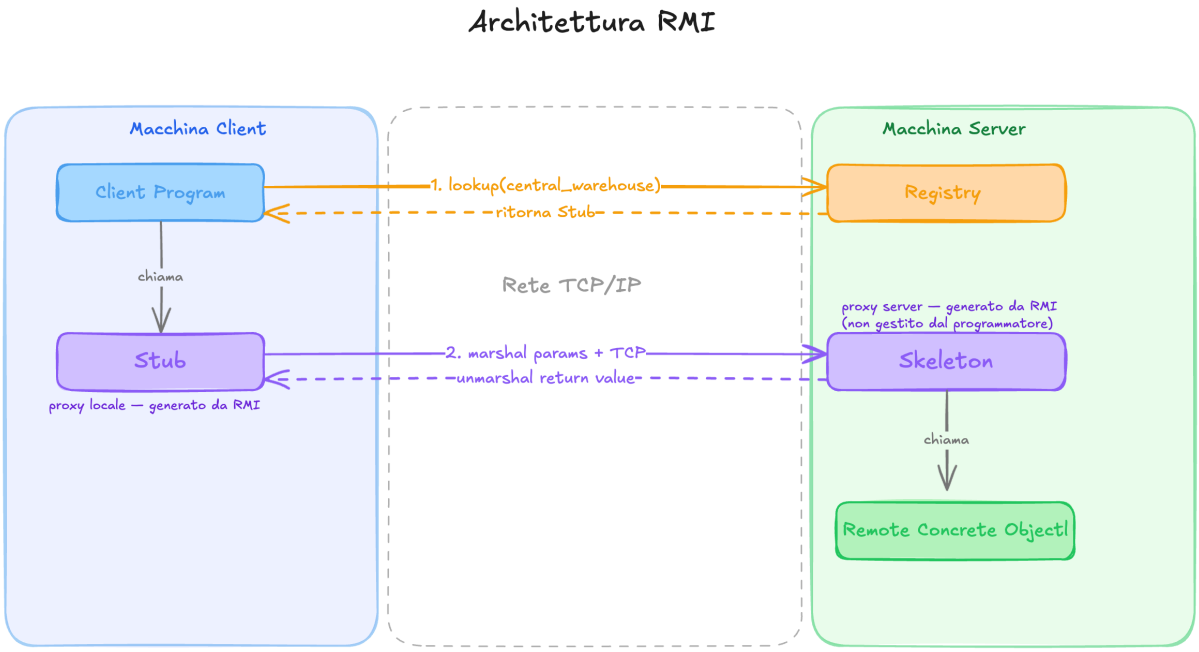


Figura 4: Architettura RMI ad alto livello

4.1 Setup Connessione

Il server pubblica nel registry RMI un oggetto di tipo `RmiConnectionEntryPoint`, che implementa `RmiConnectionService` e viene registrato con il nome `MesosServer`. Il registry funge da name service: permette al client di scoprire l'entry point del server senza dover conoscere lo stub direttamente.

```
RmiConnectionService rmiEntryPoint = new RmiConnectionEntryPoint(controller, router);
```

```
LocateRegistry.createRegistry(port);
Registry registry = LocateRegistry.getRegistry(port);
registry.rebind("MesosServer", rmiEntryPoint);
```

Il client ottiene lo stub dell'entry point tramite lookup:

```
String registryUrl = "rmi://" + serverHost + ":" + rmiPort + "/MesosServer";
RmiConnectionService connectionService =
        (RmiConnectionService) Naming.lookup(registryUrl);
```

Prima di chiamare `connect(...)`, il client esporta il proprio callback RMI, `VirtualServerRMI`: è l'oggetto remoto che il server chiamerà per consegnare notifiche al client.

```
this.callbackReceiver = new VirtualServerRMI(clientView, exportCallback);
this.serverStub = connectionService.connect(this.callbackReceiver);
this.callbackReceiver.attachServerProxy(this);
```

La chiamata a `connect(...)` arriva sul server in `RmiConnectionEntryPoint`, che riceve il callback e crea una `VirtualViewRMI` dedicata a quella sessione:

```
public RmiServerService connect(RmiClientService clientCallback) throws
    RemoteException {
    return new VirtualViewRMI(this.serverConnectionLifecycle, clientCallback, this.
    router);
}
```

`VirtualViewRMI` è la sessione remota del singolo client lato server: registra la connessione e conserva il callback ricevuto.

```
this.clientCallback = clientCallback;
this.sessionId = this.serverConnectionLifecycle.registerConnection(this);
```

Il valore restituito da `connect(...)` è uno stub di tipo `RmiServerService`, che il client userà d'ora in poi per inviare comandi — senza più passare dal registry.

4.2 Scambio messaggi

Client → server: `ServerProxyRMI` invoca un metodo sullo stub `RmiServerService`, la chiamata arriva in `VirtualViewRMI.handleClientMessage(...)`. Il messaggio viene ricevuto come oggetto Java e inoltrato direttamente al router applicativo:

```
this.serverStub.handleClientMessage(message);
```

```
ClientPayload payload = clientMessage.payload;
this.router.routeMessage(this.sessionId, payload);
```

Server → client: il model notifica `VirtualViewRMI` tramite l'Observer pattern. `VirtualViewRMI` invoca il metodo sul callback remoto del client:

```
this.clientCallback.handleMessage(
        new ServerMessage(ServerMessageType.AUTHENTICATION_RESULT,
    authenticationResultPayload)
);
```

La chiamata arriva in `VirtualServerRMI.handleMessage(...)`, che la inoltra al `MessageRouter` della UI:

```
this.messageRouter.routeMessage(message);
```

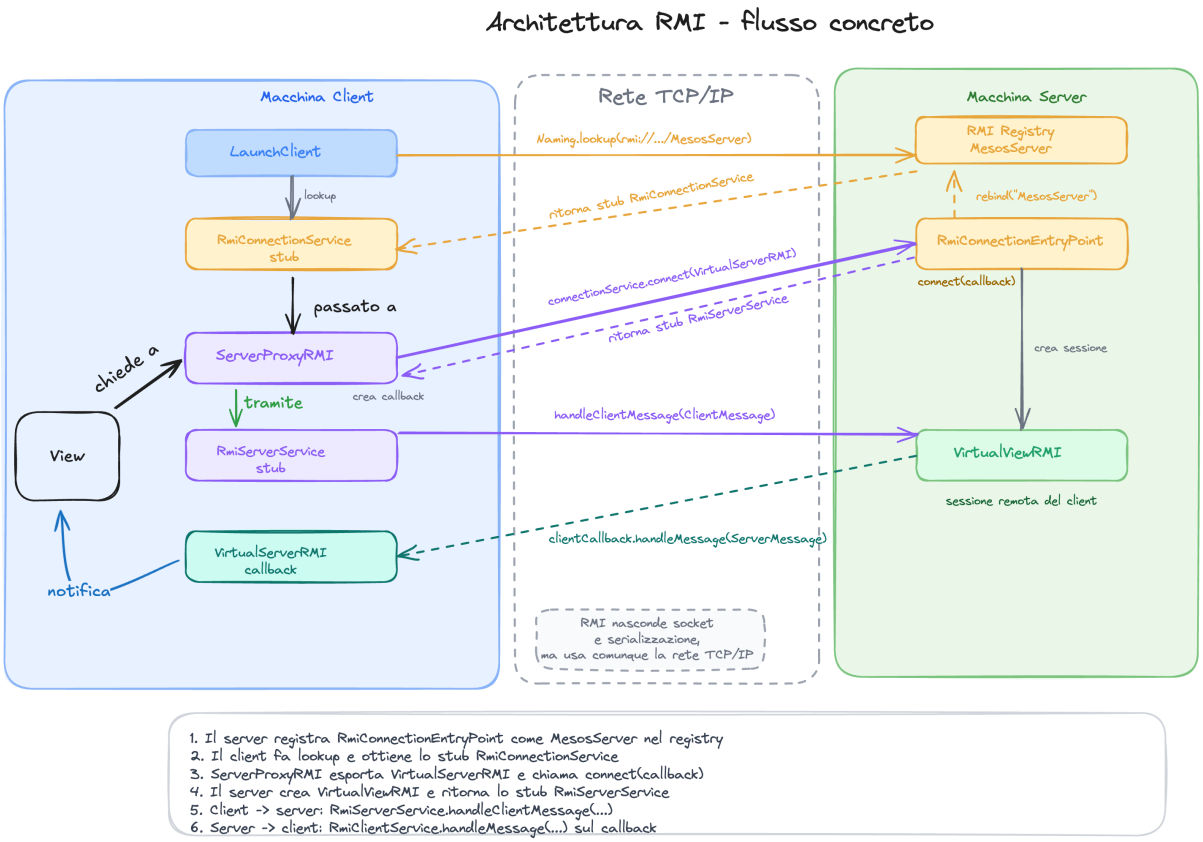


Figura 5: Flusso RMI

In sintesi il flusso concreto RMI è:

- il server registra `RmiConnectionEntryPoint` nel registry come `MesosServer`;
- il client fa lookup e ottiene lo stub `RmiConnectionService`;
- il client esporta `VirtualServerRMI` come callback e chiama `connect(callback)`;
- il server crea una `VirtualViewRMI` dedicata e restituisce lo stub `RmiServerService`;
- client → server: `ServerProxyRMI` invoca `VirtualViewRMI` tramite stub;
- server → client: `VirtualViewRMI` invoca `VirtualServerRMI` tramite callback.

5 Sequence Diagrams Commentati

In questa sezione vengono commentati alcuni sequence diagram che aiutano a comprendere il flusso concreto di messaggi tra client e server, sia in TCP che in RMI.

5.1 Boot: avvio del server e connessione dei client

Il diagramma mostra le tre fasi dell'avvio del sistema: l'inizializzazione del server, la connessione di un client TCP e la connessione di un client RMI.

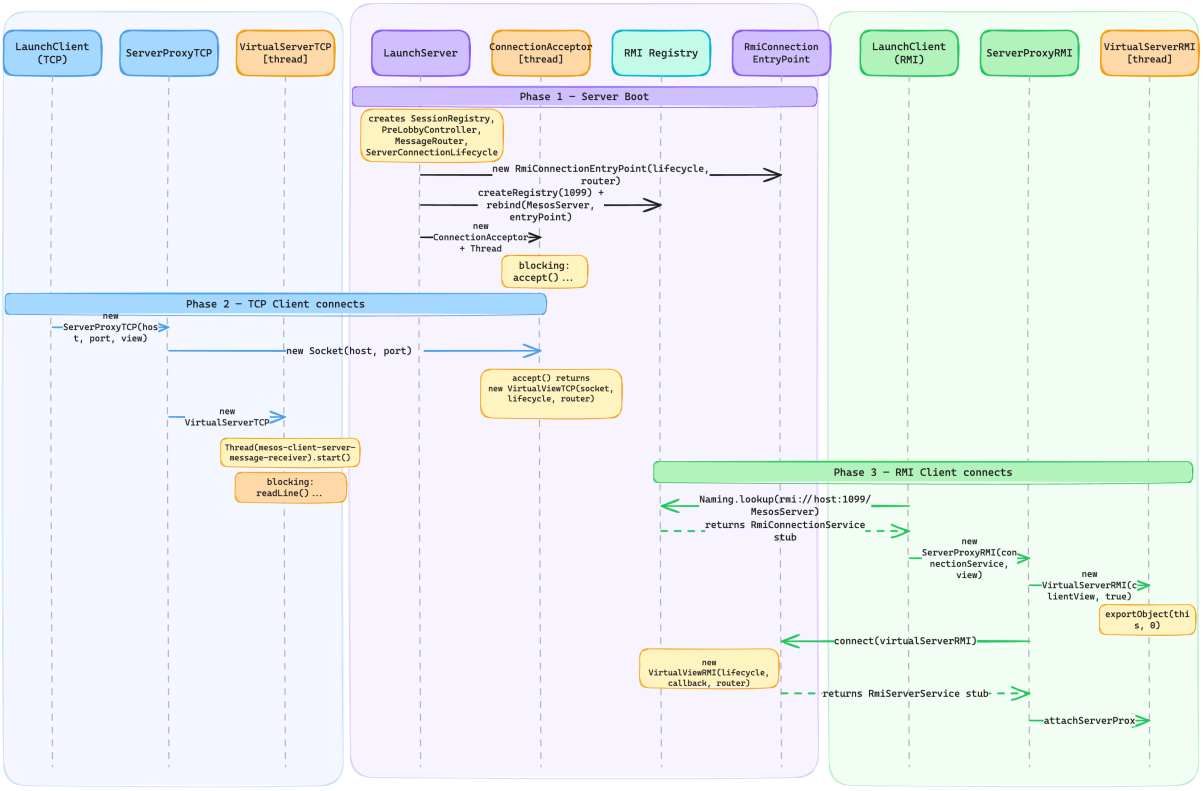


Figura 6: Sequence diagram — Boot: avvio del server e connessione TCP/RMI

Fase 1 — Avvio del server (`LaunchServer`)

`LaunchServer` inizializza i componenti del server nel seguente ordine:

1. Crea `ServerSessionRegistry`, il registro centralizzato delle sessioni di rete, delle lobby e delle partite.
2. Crea `PreLobbyController` e lo avvia su un thread dedicato. Il controller si mette subito in ascolto sulla sua coda di comandi.
3. Crea `MessageRouter` (server-side): il componente che, ricevuto un `ClientPayload`, consulta il registry per determinare la fase della sessione e instrada il messaggio al controller corretto.
4. Crea `ServerConnectionLifecycle`: gestisce gli eventi di ingresso e uscita di un client (registrazione, deregistrazione, reconnect slot).
5. Pubblica `RmiConnectionEntryPoint` nel registry RMI come `MesosServer`. Da questo momento i client RMI possono richiedere una connessione.
6. Avvia `ConnectionAcceptor` su un thread dedicato (`mesos-connection-acceptor`), che entra in un ciclo bloccante su `serverSocket.accept()`.

Fase 2 — Connessione via TCP

`LaunchClient(TCP)` crea una nuova istanza di `ServerProxyTCP`, il cui costruttore coordina tutto l'handshake lato client:

```
this.socket = new Socket(serverHost, port); // TCP handshake
this.writer = new PrintWriter(this.socket.getOutputStream(), true);
this.virtualServer = new VirtualServerTCP(this.socket, clientView, this);
this.virtualServerThread = new Thread(virtualServer,
        "mesos-client-server-message-receiver");
this.virtualServerThread.start();
```

Sul server, `ConnectionAcceptor` sblocca da `accept()`, crea un `VirtualViewTCP` dedicato al client e lo avvia su thread proprio:

```
Socket clientSocket = this.serverSocket.accept();
VirtualViewTCP virtualView = new VirtualViewTCP(
        clientSocket, this.serverConnectionLifecycle, this.router);
new Thread(virtualView,
        "mesos-virtual-view-" + clientSocket.getPort()).start();
```

Al termine dell'handshake, per ogni client TCP sono attivi due thread simmetrici: `VirtualViewTCP` (server) e `VirtualServerTCP` (client), entrambi bloccati su `readLine()` in attesa di messaggi.

Fase 3 — Connessione via RMI

`LaunchClient(RMI)` ottiene lo stub di `RmiConnectionService` tramite `Naming.lookup(url)`, poi istanzia `ServerProxyRMI`. Il costruttore orchestra la sequenza di setup:

1. Crea e pubblica `VirtualServerRMI` come oggetto remoto (il callback che il server userà per rispondere al client).
2. Chiama `connect(callback)` su `RmiConnectionEntryPoint` (lato server). Il server crea e restituisce un `VirtualViewRMI` come stub `RmiServerService`.
3. Conserva lo stub in `this.serverStub`: da ora in poi tutti i comandi del client passeranno da qui, senza toccare il registry.
4. Chiama `attachServerProxy(this)` su `VirtualServerRMI` per completare il collegamento bidirezionale.

5.2 ChooseCard via TCP

Il diagramma mostra il ciclo completo di un'azione di gioco — la scelta di una carta — su connessione TCP. Il punto centrale da evidenziare è dove il *return* della chiamata avviene rispetto alla elaborazione effettiva sul server.

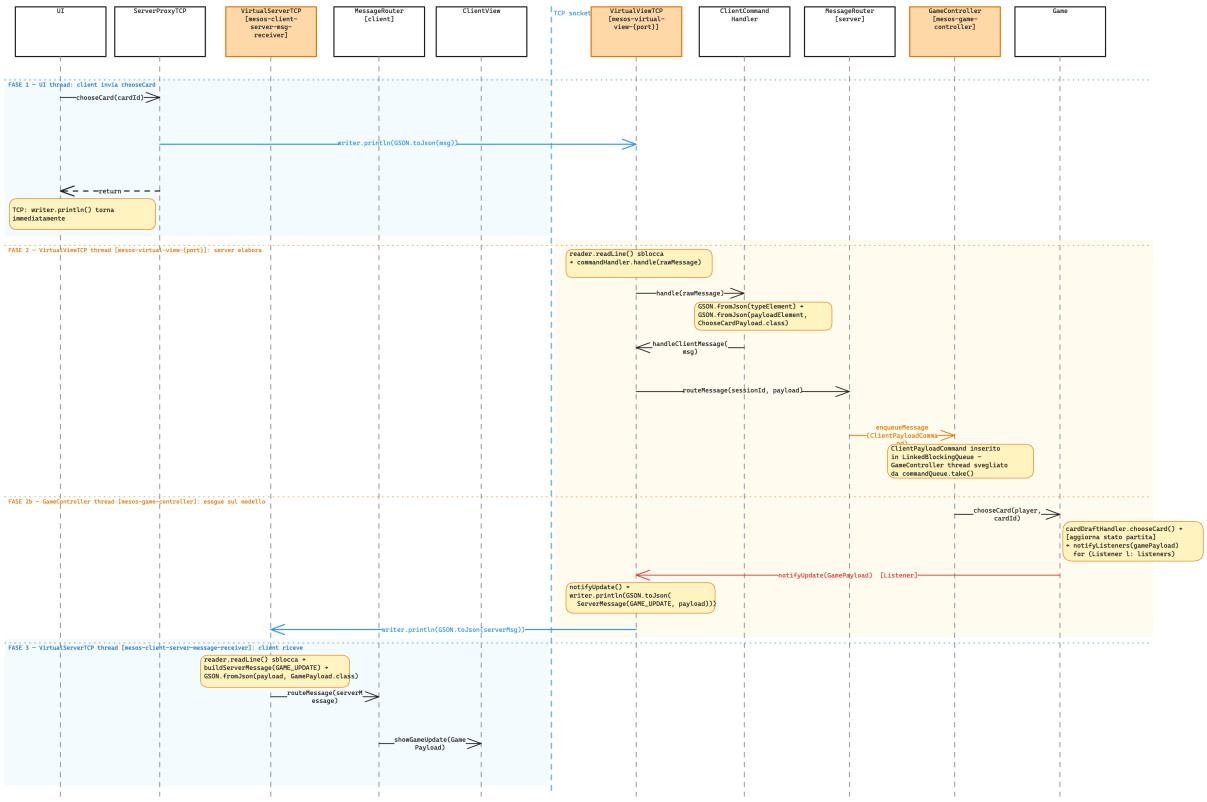


Figura 7: Sequence diagram — ChooseCard via TCP

Fase 1 — Invio del comando

La UI che può essere sia testuale che grafica, chiama `ServerProxyTCP.chooseCard(cardId)`. Il metodo crea un `ChooseCardPayload`, lo insersice in un `ClientMessage` e serializza il tutto con GSON:

```
String serialized = GSON.toJson(message);
this.writer.println(serialized); // scrive sul socket e ritorna subito
```

`writer.println()` **ritorna immediatamente**.

Fase 2 — Elaborazione sul server

Sul server, il thread `mesos-virtual-view` era bloccato su `reader.readLine()`. Quando il JSON arriva, si sblocca e delega a `ClientCommandHandler.handle(rawMessage)`:

```
ClientMessageType type = ClientMessageType.valueOf(typeElement.getAsString());
// ...
ClientPayload payload = GSON.fromJson(payloadElement, ChooseCardPayload.class);
ClientMessage clientMessage = new ClientMessage(type, payload);
this.virtualView.handleClientMessage(clientMessage);
```

`VirtualViewTCP.handleClientMessage()` consegna il payload al `MessageRouter` (server-side), che consulta il `SessionRegistry` per determinare la fase della sessione (GAME) e inserisce un `ClientPayloadCommand` nella coda del `GameController`:

```
gameOrchestrator.enqueueMessage(new ClientPayloadCommand(sessionId, payload));
```

A questo punto il thread di `VirtualViewTCP` ha finito: la `enqueueMessage` è non bloccante (torna subito) e il thread torna in attesa della prossima riga sul socket.

Il thread `mesos-game-controller` era bloccato su `commandQueue.take()`. Sbloccato, estrae il comando, risolve il `Player` dalla sessione e delega al modello:

```
this.game.chooseCard(player, p.cardId);
```

Fase 2b — Notifica via Observer (`Listener`)

Dopo aver aggiornato lo stato, `Game` chiama `notifyListeners()`:

```
for (Listener listener : new ArrayList<>(this.listeners)) {
    listener.notifyUpdate(payload);
}
```

Ogni `VirtualViewTCP` è registrato come `Listener`. La sua implementazione serializza il payload e scrive sul socket in direzione server→client:

```
// VirtualViewTCP.notifyUpdate()
this.writer.println(GSON.toJson(
        new ServerMessage(ServerMessageType.GAME_UPDATE, gamePayload)));
```

Fase 3 — Ricezione della risposta (client receiver thread)

Sul client, il thread `mesos-client-server-message-receiver` era bloccato su `reader.readLine()`. Sbloccato, `VirtualServerTCP` deserializza il `ServerMessage` e lo passa al `MessageRouter` (client-side):

```
// buildServerMessage() seleziona la classe concreta dal type
GamePayload gamePayload = GSON.fromJson(payload, GamePayload.class);
ServerMessage message = new ServerMessage(type, gamePayload);
this.messageRouter.routeMessage(message);
```

Il `MessageRouter` del client chiama infine `clientView.showGameUpdate(gamePayload)`, che aggiorna l'interfaccia utente.

5.3 ChooseCard via RMI

Il diagramma mostra lo stesso ciclo di `chooseCard`, questa volta su connessione RMI. Mette in evidenza le differenze strutturali rispetto al caso TCP.

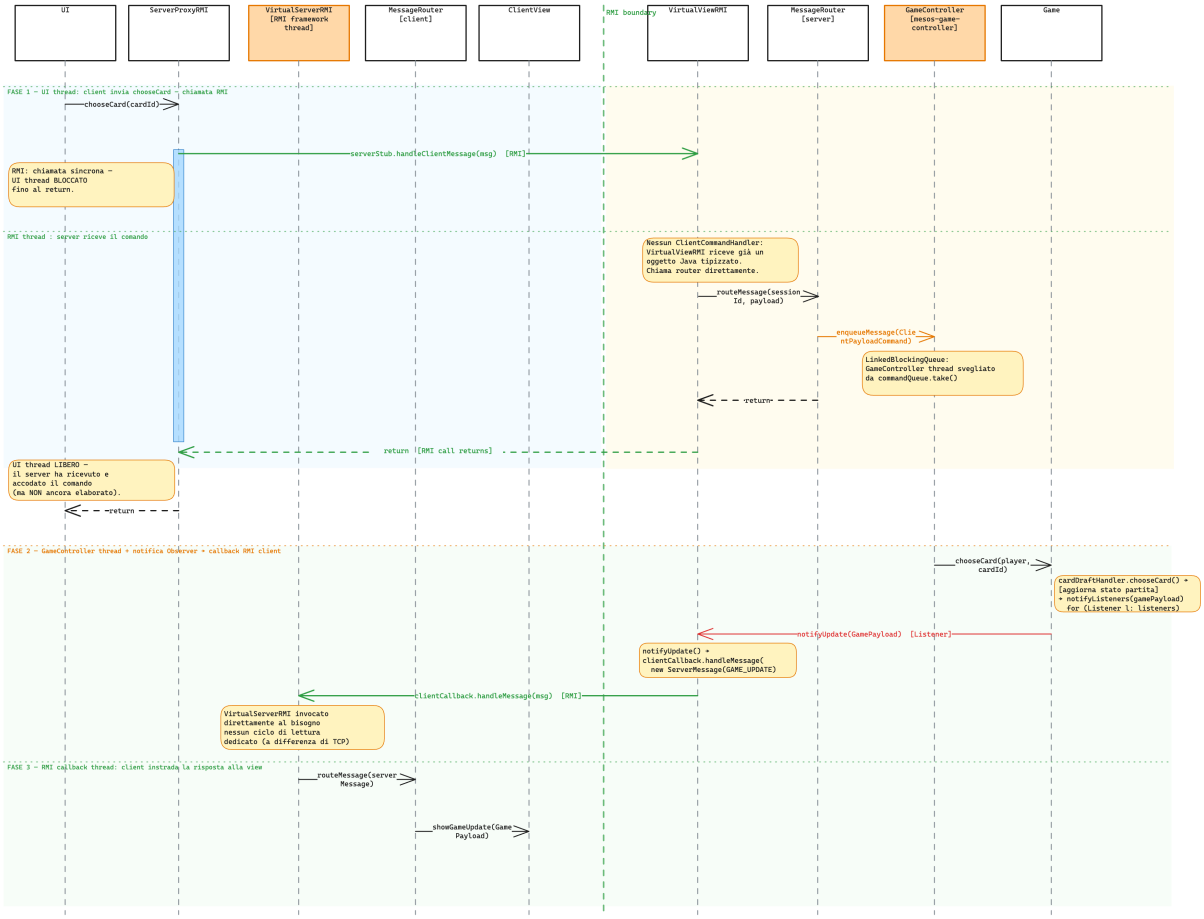


Figura 8: Sequence diagram — ChooseCard via RMI

In questo sequence diagram si può apprezzare come l'implementazione dei controller su thread con raccolta dei comandi da coda, ci aiuti sensibilmente nel ridurre il gap di latenza tra RMI e TCP. Difatti il metodo che inoltra il comando a partire su TCP termina quasi subito, come si vedrà non è lo stesso per RMI, dove abbiamo che il metodo non torna fintanto che il server non ha ricevuto e accodato il comando.

Fase 1 — Invio del comando

La UI chiama `ServerProxyRMI.chooseCard(cardId)`. Il metodo crea un `ChooseCardPayload`, lo avvolge in un `ClientMessage` e invoca il metodo remoto sullo stub:

```
this.serverStub.handleClientMessage(message);
```

A differenza di TCP, questa chiamata **blocca il thread UI** fino a quando il metodo sul server non ritorna. Non c'è serializzazione GSON esplicita: i payload vengono passati come oggetti Java tramite la serializzazione standard di Java RMI.

Fase 1b — Elaborazione lato server

Sul server, la chiamata arriva direttamente in `VirtualViewRMI.handleClientMessage(ClientMessage clientMessage)`. A differenza del caso TCP, non esiste un `ClientCommandHandler` che deserializza la stringa JSON: il messaggio è già un oggetto Java tipizzato. `VirtualViewRMI` consegna il payload al `MessageRouter` direttamente:

```
// VirtualViewRMI.handleClientMessage()
ClientPayload payload = clientMessage.payload;
this.router.routeMessage(this.sessionId, payload);
```

`MessageRouter` inserisce un `ClientPayloadCommand` nella coda del `GameController`, esattamente come nel caso TCP. Quando `routeMessage()` ritorna, anche la chiamata RMI ritorna: il thread UI sul client si sblocca e `ServerProxyRMI.chooseCard()` restituisce il controllo all'UI.

> Si noti come il thread UI si sblocca dopo che il server ha *ricevuto e accodato* il comando, non dopo che il modello lo ha elaborato. Il `GameController` thread processerà il comando in modo asincrono, esattamente come nel caso TCP.

Fase 2 — Elaborazione del modello e notifica

Il `GameController` thread processa il comando dalla coda e invoca `game.chooseCard(player, cardId)`, identico al caso TCP. `Game.notifyListeners()` itera i listener: ogni `VirtualViewRMI` registrato riceve `notifyUpdate(gamePayload)`.

`VirtualViewRMI.notifyUpdate()` costruisce il `ServerMessage` e lo consegna al client tramite il callback remoto:

```
// VirtualViewRMI.notifyUpdate()
this.clientCallback.handleMessage(
        new ServerMessage(ServerMessageType.GAME_UPDATE, gamePayload));
```

Questa è un'altra chiamata RMI, ma in direzione inversa: il *server* chiama un metodo sul *client*.

Fase 3 — Ricezione della risposta

La chiamata arriva in `VirtualServerRMI.handleMessage(ServerMessage)`. Non esiste un ciclo `readLine()` come in TCP: il framework RMI gestisce l'arrivo della chiamata remota in modo trasparente.

```
// VirtualServerRMI.handleMessage()
this.messageRouter.routeMessage(message); // -> clientView.showGameUpdate()
```

Tabella differenze TCP vs RMI (per esempio su chooseCard):

Aspetto	**TCP**	**RMI**
Thread UI dopo invio	libero subito	bloccato fino al return
Serializzazione	GSON esplicita (JSON su socket)	RMI Library
`ClientCommandHandler`	sì (deserializza JSON)	no (oggetto già tipizzato)
Thread di ricezione client	dedicato (`readLine()` loop)	RMI Library
Callback server→client	`writer.println()`	`clientCallback.handleMessage()`